

Creating Song List III

Week 07.





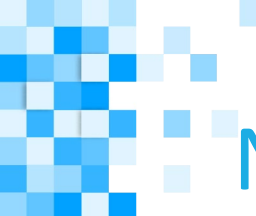
Overview

- After creating the screen interface and song data, you will learn how to handle the user selection in this workshop.



Objectives

- Demonstrate how to handle event;
- Demonstrate how to search through an array;
- Demonstrate how to write methods and parameters.
- Demonstrate how to use Android Intent to send data to other activity.



Note

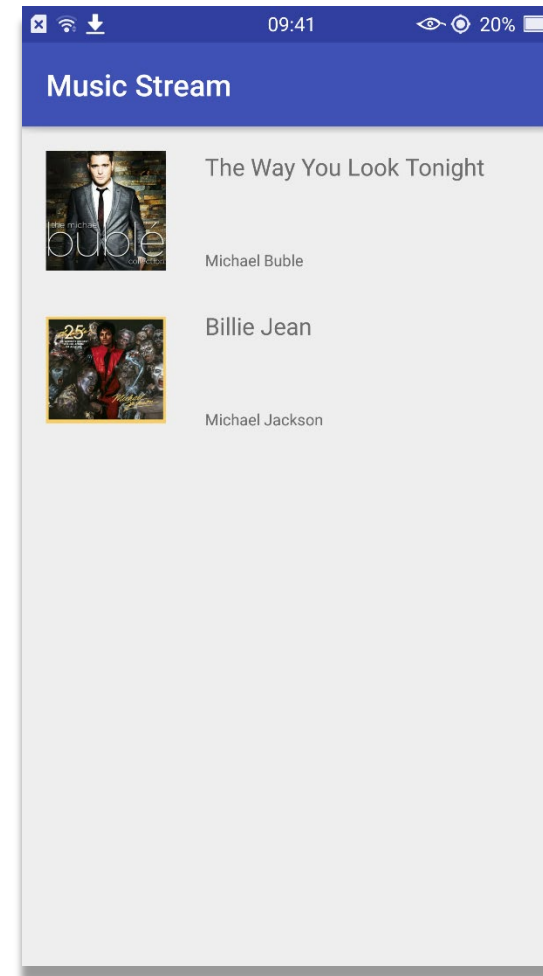
- Throughout this document, the acronym “**app**” will be used to refer to **mobile application**.



HANDLING SONG SELECTION

How to Handle a Selected Song?

- Let's look at the song list that you've created.
- If the user touch on the thumbnail to select the song, which is created using `ImageButton`, how would the app play the selected song? (This was discussed in Week 5)
- Some kind of data needs to be passed to your codes in **MainActivity.java** so that your codes can recognize the imagebutton being tapped.
- So how do you get the data?
- The answer will be explained in the next few slides.





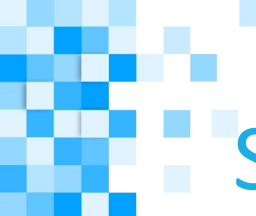
How to Handle a Selected Song?

- Let's re-visit the steps that you need to do to handle a song selection. The steps have been mentioned in the slides of Creating Song List I.
- Take a look at the codes in the next slides. We can deduce to these 3 steps.

Step 1 : Get the ID of the selected song.

Step 2: Search for the selected song based on the ID so that all information/data of the song can be retrieved from a song list.

Step 3: Send the song data to the player screen to be played.



Step 1 : Get the ID of the selected song.

This was covered in Week 5.

Recap : When any of the imagebutton is tapped, that particular ImageButton instance (with its properties) is being passed into the “myview” argument (aka variable).

`myView.getId()` – will then execute to give you the assigned integer for that ImageButton (Android will assigned an interger number to every instance when it is running. We would not be able to know this “number” in advance as it is managed by the Android OS.).

`getResources().getResourceEntryName(myView.getId())` - this next line of code is making use of an available Android api (aka method) to help us discover what is the given String ID (for eg. “S1001, “S1002”) whenever we passed in the numeric integer that we obtained in the previous step. It’s a bit like “reverse engineering”

Finally what we will get in the variable “resourceId” will be the string “S1002” (assuming it is the second ImageButton that was being tapped. You can Log.d to verify and test out by tapping different ImageButtons)

```
public void handleSelection(View myView) {  
    String resourceId = getResources().getResourceEntryName(myView.getId());  
}  
}
```


Step 2: Searching Through the Song List ...

Here is the code of the `searchSongById` method. Add the method to `SongCollection.java`.

This method will give the ability to return a song object to the caller based on a given string

The caller to this method will pass (for eg. string "S1002") into the argument "id".

The for loop will iterate and check through each item in the array to see if any of the item also has the same id (eg "S1002")

If found and it WILL be found unless you had named the id of the imagebuttons wrongly or created the array with typo id. This method will then return the index position of that item in the array back to the caller. Spend some time to understand the loop and the if clause in the loop.

For eg. If it is the imagebutton "S1002" that is being tapped and "S1002" is being passed into the method, the method will return the integer **1** to the caller as the song "S1002" occupy the 2nd position in the array (remember array starts the counting from 0 instead of 1).

Theoretically, line 42 should never be executed for any scenario unless you had named the ImageButton wrongly or had typo in the Array

```
4 private Song songs[] = new Song[3];
5
6 public SongCollection(){
7
8     Song theWayYouLookTonight = new Song( id: "S1001",
9         title: "1. The Way You Look Tonight",
10        artiste: "Michael Buble",
11        fileLink: "https://p.scdn.co/mp3-preview/a5b8972e764025020625bbf9c1c2bbb06e394a60?cid=2afe87a64b0042dabf51f37318616965",
12        songLength: 4.66,
13        R.drawable.michael_buble_collection);
14
15     Song billieJean = new Song( id: "S1002",
16        title: "2. Billie Jean",
17        artiste: "Michael Jackson",
18        fileLink: "https://p.scdn.co/mp3-preview/4eb779428d40d579f14d12a9daf98fc66c7d0be4?cid=2afe87a64b0042dabf51f37318616965",
19        songLength: 4.9,
20        R.drawable.billie_jean);
21
22     Song ed = new Song( id: "S1003",
23        title: "3. One",
24        artiste: "Ed Sheeran",
25        fileLink: "https://p.scdn.co/mp3-preview/daa8679253ba20620db6e1db9c88edfcf1f4069f?cid=2afe87a64b0042dabf51f37318616965",
26        songLength: 4.9,
27        R.drawable.photograph);
28
29     songs[0] = theWayYouLookTonight;
30     songs[1] = billieJean;
31     songs[2] = ed;
32 }
33
34 public int searchSongById(String id){
35
36     for(int index=0; index < songs.length; index++){
37         Song tempSong = songs[index];
38         if(tempSong.getId().equals(id)){
39             return index;
40         }
41     }
42     return -1;
43 }
```

Step 3: Send the song data to the player screen to be played.

Complete the 2 methods by following the codes below.

Basically the sendDataToActivity method will now received the array index position of the ImageButton that was tapped. It will make use of the Intent object api to pack this information and send it over to another Android activity (PlaySongActivity – to be created next).

Make sure you had also wired up all the ImageButtons so that it will call the handleSelection by using the “onclick” eventListener when being tapped. Refer to Week 5 lab.

```
public void sendDataToActivity(int index){  
    Intent intent = new Intent( packageContext: this, PlaySongActivity.class);  
    intent.putExtra( name: "index", index);  
    startActivity(intent);  
}
```

Further explanation in the next few slides

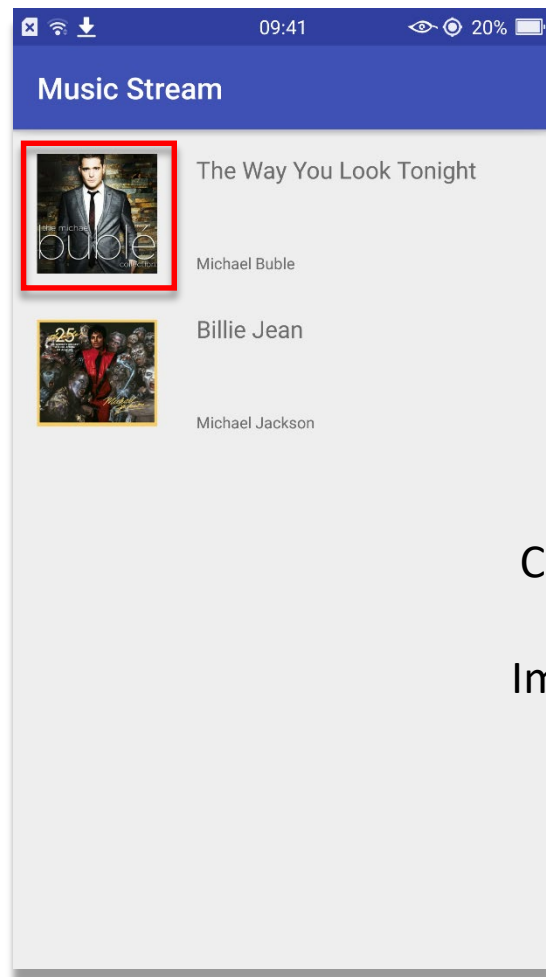
```
public void handleSelection(View myView) {  
    String resourceId = getResources().getResourceEntryName(myView.getId());  
    int currentArrayIndex = songCollection.searchSongById(resourceId);  
    Log.d( tag: "temasek", msg: "The index in the array for this song is :" + currentArrayIndex);  
    sendDataToActivity(currentArrayIndex);  
}
```

Calling the method that we had implemented in the previous slide

Logging out for checking and trouble-shooting purposes

Calling the above method by passing in the position of the song in the array

Step 3: Send the song data to the player screen to be played



Intent



Contain the integer 0
as it was the first
Imagebutton that was
being tapped



Step 3: Send the song data to the player screen to be played

This method does not need to return any result so we set the return type as `void`.

This method requires the caller to also pass in an integer (for our case, that integer will represent the array position of that song)

This will appear as an error as you might not have created the `PlaySongActivity` yet. Hence you will not be able to compile and run your app

```
public void sendDataToActivity(int index){  
    Intent intent = new Intent(packageContext.this, PlaySongActivity.class);  
    intent.putExtra(name: "index", index);  
    startActivity(intent);  
}
```

This part is the `value` which is tied to the key.

The `startActivity` is a built-in method in the Android system to be used to launch an activity. It has `Intent` as a parameter. After we store the song information to the `intent`, we just pass it to `startActivity` and it will launch the `PlaySongActivity`.

This part is the `key` which is of `String` type. It is up to you to name it. When you retrieve the value in the destination screen/activity, you will get the value using this key. You can send as many pair of values as you wish. Here we are only sending one set of data which is the array position of the song. It is up to you to send over any number of set of data



Summary

- You have now successfully completed the song list and perform searching through it!
- However you will not be able to compile and run as you do not have the PlaySongActivity class
- In the next lesson, you will learn how to play the selected song.



Supplementary Materials

Topic	Source
Java Method/Function	https://www.learnjavaonline.org/en/Functions https://mathbits.com/MathBits/Java/Methods/Lesson1.htm http://www.horsesnw.com/articles/JavaMethod.htm
Java Loop	https://www.learnjavaonline.org/en/Loops
Android Intent	https://developer.android.com/training/basics/firstapp/starting-activity.html#BuildIntent http://www.vogella.com/tutorials/AndroidIntent/article.html#what-are-intents

THE END



COMT